

MuleSoft Integration Dashboard

A full-stack monitoring and management dashboard for MuleSoft Anypoint Platform, built with React 18, Vite, Tailwind CSS (frontend) and Node.js/Express (backend). It proxies Anypoint Platform REST APIs to provide a unified view of your integrations across all Business Groups, environments, and deployment targets.

Table of Contents

1. [Overview](#)
 2. [Tech Stack](#)
 3. [Project Structure](#)
 4. [Getting Started](#)
 - [Prerequisites](#)
 - [Environment Variables](#)
 - [Installation](#)
 - [Running the Application](#)
 5. [Features & Use Cases](#)
 - [Authentication](#)
 - [Dashboard \(Home\)](#)
 - [Applications](#)
 - [Application Detail](#)
 - [API Manager](#)
 - [Exchange Assets](#)
 - [Ping Test](#)
 - [Business Groups](#)
 - [Environments](#)
 6. [Architecture Overview](#)
 7. [Backend API Routes Reference](#)
 8. [Ping Test Logic](#)
 9. [Known Limitations](#)
-

Overview

The MuleSoft Integration Dashboard gives platform operators, integration developers, and architects a single-pane-of-glass view of their Anypoint Platform estate. It connects directly to the Anypoint Platform APIs using your credentials (username/password) and surfaces:

- Real-time application status across CloudHub 1.0 and CloudHub 2.0
- API Manager instances, applied policies, and consumer contracts
- Anypoint Exchange asset catalogue with business-group filtering
- Config Property Server (CPS) values per application
- A dedicated Ping Test module for live health-check verification
- Organization-wide metrics and usage charts

Tech Stack

Layer	Technology
Frontend	React 18, Vite, React Router v6, Tailwind CSS
Backend	Node.js, Express 4, Axios, express-session
Platform APIs	Anypoint ARM, API Manager, Exchange, CPS, Metrics, Environments, Organizations
Charts	Recharts
HTTP Client	Axios (backend proxy), Fetch API (frontend)

Project Structure

```
None
mulesoft-dashboard/
├── backend/
│   ├── package.json
│   └── src/
│       ├── server.js                # Express entry point, route registration
│       └── routes/
│           ├── auth.js              # POST /api/login, GET /api/logout, GET
│           ├── applications.js      # CloudHub 1.0 & 2.0 app listing, actions,
│           ├── apis.js              # API Manager instances, policies,
│           ├── contracts, alerts
│           └── exchange.js          # Exchange asset search
```

```

|           |— environments.js      # Environment listing
|           |— organizations.js    # Business group hierarchy
|           |— metrics.js          # Application metrics aggregation
|           |— cps.js              # Config Property Server values
|           |— health.js           # POST /api/ping - live health-check
endpoint
|— frontend/
|   |— package.json
|   |— vite.config.js
|   |— src/
|       |— App.jsx                 # React Router route definitions
|       |— components/
|           |— Sidebar.jsx         # Navigation sidebar
|           |— PingTestPanel.jsx   # Single-app ping panel (used in detail
page)
|   |— PingResultCard.jsx         # Result card component
|   |— BulkPingModal.jsx         # Bulk ping modal (launched from
Applications page)
|       |— pages/
|           |— DashboardPage.jsx   # Metrics, charts, environment summary
|           |— ApplicationsPage.jsx # Full app list with bulk operations
|           |— ApplicationDetailPage.jsx # 6-tab deep-dive per application
|           |— ApiManagerPage.jsx  # API instances, policies, contracts
|           |— ExchangePage.jsx    # Exchange asset browser
|           |— PingTestPage.jsx    # Dedicated ping-test workspace
|— README.md

```

Getting Started

Prerequisites

- Node.js 18+ and npm
- Access to an Anypoint Platform account (username + password)
- Network access to anypoint.mulesoft.com (or your Anypoint PCE/GOV endpoint)

Environment Variables

Create a `.env` file in `backend/` (copy from `.env.example` if provided):

```

None
# Port the Express server listens on

```

```
PORT=5000
```

```
# Session secret - change to a random string in production  
SESSION_SECRET=your-secret-here
```

```
# Anypoint Platform base URL (default shown)  
ANYPOINT_BASE_URL=https://anypoint.mulesoft.com
```

The frontend Vite dev-server proxies all `/api` requests to `http://localhost:5000`, so no frontend `.env` is required for local development.

Installation

```
Shell
```

```
# Install backend dependencies
```

```
cd backend
```

```
npm install
```

```
# Install frontend dependencies
```

```
cd ../frontend
```

```
npm install
```

Running the Application

```
Shell
```

```
# Start the backend (from project root)
```

```
cd backend
```

```
npm start          # or: node src/server.js
```

```
# Start the frontend dev server (separate terminal)
```

```
cd frontend
```

```
npm run dev
```

Open <http://localhost:5173> in your browser.

For production, build the frontend and serve it as static files from Express:

```
Shell
```

```
cd frontend
```

```
npm run build
```

```
# Copy dist/ into backend/public/ or configure Express to serve it
```

Features & Use Cases

Authentication

Route: `/login`

- Enter your Anypoint Platform **username** and **password**.
- The backend calls the Anypoint `POST /accounts/login` API, retrieves a bearer token, and stores it server-side in an Express session.
- All subsequent API calls from the dashboard use this session token — credentials are never stored in the browser.
- A `GET /api/me` call validates the session on every page load to ensure the user is still authenticated.
- Logout clears the server-side session and redirects to `/login`.

Use case: Secure, session-based access to the Anypoint Platform without exposing credentials to the browser.

Dashboard (Home)

Route: `/`

Backend routes used: `/api/environments`, `/api/organizations`, `/api/metrics/summary`, `/api/applications/summary`

The Dashboard provides an at-a-glance operational view:

Widget	Description
Total Applications	Count of deployed apps across selected BG / all BGs
Running / Stopped	Live status breakdown
Total APIs	Count of API Manager instances
Environment Breakdown	Bar or pie chart of apps per environment
Application Status Chart	Running vs Stopped vs Unknown donut chart

Widget	Description
Metrics Summary	Aggregated message throughput and error rates

Business Group selector — defaults to **All Organizations**, which aggregates data across every Business Group your account has access to.

Use case: Morning health-check for platform engineers; executive-level integration estate overview.

Applications

Route: </applications>

Backend routes used: </api/applications>, </api/applications/summary>

A paginated, filterable table of all Mule applications across CloudHub 1.0 and CloudHub 2.0.

Columns

Column	Description
☒	Multi-select checkbox
Application Name	Links to Application Detail page
Type	CH1 / CH2 badge
Status	STARTED / STOPPED / FAILED with colour coding
Environment	Deployment environment name
Workers / Replicas	Resource allocation
Runtime Version	Mule runtime version
Last Modified	Timestamp of last deployment

Key Features

- **Business Group selector** — Filter apps by a specific BG or view **All Organizations** (aggregates every BG in parallel).

- **Environment filter** — Narrow down by Sandbox, Production, UAT, etc.
- **Status filter** — Running, Stopped, or All.
- **Search** — Full-text filter on application name.
- **Multi-select with floating rows** — Selected rows automatically float to the top of the table so your selection is always visible even when scrolling through hundreds of apps.
- **Bulk Ping** — Select one or more apps and click **Ping Selected** to run live health checks. Results open in the Ping Test page pre-populated with data.
- **Start / Stop / Restart** — Per-row action buttons call the ARM API to control application lifecycle.

Use case: Daily operations review; finding failing apps across all BGs; selecting a set of apps for bulk health verification before a release window.

Application Detail

Route: `/applications/:id`

Tabs: Overview · Properties · Infra & Config · CPS Config · Ping Test · Raw JSON

A deep-dive view for a single application with six tabs:

Tab 1 — Overview

- Status badge, runtime version, region, workers/replicas
- Last deployment timestamp and deployment model (CH1/CH2)
- Direct link to Anypoint Runtime Manager

Tab 2 — Properties

- All application deployment properties (environment variables / property placeholders) surfaced from the ARM deployment descriptor.
- Useful for verifying property placeholder values without logging into Anypoint Studio or Runtime Manager.

Tab 3 — Infra & Config

- Worker size / replica configuration
- Auto-scaling settings (if applicable)
- Persistent queues, persistent object store flags
- Monitoring and logging configuration
- Network settings (VPC, static IPs)

Tab 4 — CPS Config

- Fetches values from the **Anypoint Config Property Server (CPS)** for this application.

- Displays key-value pairs with environment context.
- Allows operators to verify runtime property values from a central location without SSH or log access.

Use case: Troubleshooting misconfigured properties; verifying CPS values match expected environment settings post-deployment.

Tab 5 — Ping Test

- Embedded single-app ping panel (`PingTestPanel` component).
- Enter optional `client_id` and `client_secret` for APIs that require client credentials.
- Runs the full endpoint discovery loop (see [Ping Test Logic](#)).
- Displays status badge, latency, active endpoint path, and response payload.

Use case: Quick connectivity check from the application detail page without switching to the Ping Test page.

Tab 6 — Raw JSON

- Displays the full raw JSON response from the ARM API for debugging purposes.
- Useful when diagnosing unexpected behaviour or capturing deployment metadata.

API Manager

Route: `/api-manager`

Backend routes used: `/api/apis`, `/api/apis/:id/policies`,
`/api/apis/:id/contracts`

Browse and inspect API instances registered in Anypoint API Manager.

Features

- **API Instance list** — Displays API name, version, status, environment, and deployment type.
- **Applied Policies** — Expands to show all policies applied to an API instance with **human-readable names** (resolved from the policy asset `template.name` or `assetId`, not raw numeric IDs).
- **Consumer Contracts** — Shows all approved client applications consuming the API, including:
 - Consumer application name
 - Client ID (with multiple field-path fallbacks to handle different API Manager response shapes)
 - Contract status (APPROVED / PENDING / REVOKED)

- SLA tier name
- **Alerts** — API-level alerting configuration.

Use case: Governance review; identifying which client apps are consuming a given API; verifying policy enforcement before a go-live.

Exchange Assets

Route: `/exchange`

Backend routes used: `/api/exchange/assets`

Browse assets published to Anypoint Exchange within your organisation.

Features

- **Business Group selector** — Filter assets by BG (scoped to `organizationId`).
- **Asset type filter** — REST API, RAML Fragment, Connector, Template, Example, OAS, WSDL, Custom.
- **Search** — Full-text search across asset names (up to 100 results per query).
- **Asset detail panel** — Click any asset to see:
 - Description, version, classifier
 - Tags and categories
 - Asset portal link
 - Contact information
- **Auto-select from navigation state** — When another page navigates to Exchange with a specific asset in `location.state`, the asset is automatically selected and the detail panel opens.

Use case: Asset discovery; verifying what API specs are published before implementing a new integration; checking connector versions in use.

Ping Test

Route: `/ping-test`

Backend route used: `POST /api/ping`

A dedicated workspace for running live HTTP health checks against deployed Mule applications.

Table Columns

Column	Description
☒	Select apps to include in a run
Application	Application name and BG
Type	CH1 / CH2
Status	ARM deployment status
Active Endpoint	Discovered endpoint path that returned a successful ping
HTTP	HTTP status code from the ping response
Latency	Round-trip time in milliseconds
▶	Run ping for this single row

Key Features

- **Selected rows float to top** — Selected applications always appear at the top of the table for visibility.
- **Run N selected / Run all** — Ping only selected apps or run all at once with a single button.
- **Expandable rows** — Click any row to expand and see:
 - Full endpoint URL attempted
 - Complete error message (if failed)
 - Response payload (first 500 characters)
 - Attempt log showing each path tried in order
- **View tested apps toggle** — Toggle between "All apps" and "Tested apps only" to focus on results.
- **Pre-loaded results** — When navigated from the Applications page Bulk Ping flow (`location.state`), results are immediately populated without re-running.
- **Client credentials** — Optional `client_id` and `client_secret` fields sent as headers for protected APIs.

Use case: Pre-release smoke testing; incident investigation; verifying connectivity after a network change or VPC update.

Business Groups

Route: `/business-groups`

Backend route used: `/api/organizations`

View the Business Group (BG) hierarchy for your Anypoint organisation.

- Displays the tree of parent and child Business Groups.
 - Shows organisation IDs (useful for constructing direct API calls).
 - **All Organizations** is a virtual node that triggers aggregation across all BGs in pages that support it (Dashboard, Applications, Ping Test).
-

Environments

Route: `/environments`

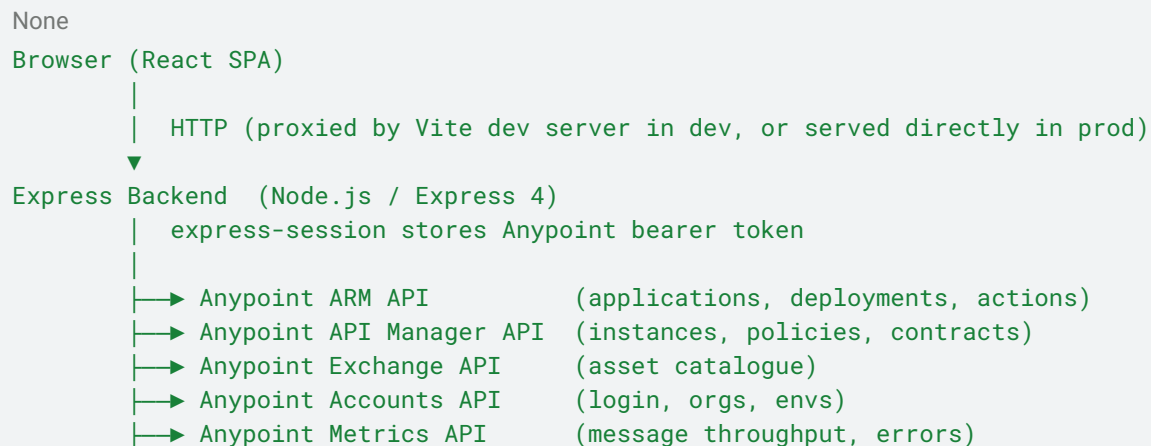
Backend route used: `/api/environments`

Lists all Anypoint environments (Sandbox, Production, Design, etc.) for your organisation.

- Displays environment name, type, and ID.
- Used by the Applications page and API Manager page for environment-scoped filtering.

Use case: Understanding available environments before deploying or filtering application lists.

Architecture Overview



```
└─> Anypoint CPS API      (config property server)
└─> Target Mule App URLs   (direct HTTP ping - health checks)
```

Authentication Flow

None

1. Browser → POST /api/login (username, password)
2. Backend → POST anypoint.mulesoft.com/accounts/login
3. Anypoint returns bearer token
4. Backend stores token in express-session (server-side)
5. All subsequent /api/* calls attach token from session
6. Browser never sees the bearer token

Data Aggregation (All Organizations)

When the user selects **All Organizations**, the backend fans out requests to each Business Group in parallel using `Promise.allSettled()`, then merges and deduplicates the results before returning them to the frontend. This allows a single page load to show the full estate without multiple manual BG switches.

Backend API Routes Reference

Auth

Method	Path	Description
POST	/api/login	Authenticate with Anypoint Platform; creates session
GET	/api/logout	Destroys session
GET	/api/me	Returns current session user info

Applications

Method	Path	Description
GET	<code>/api/applications</code>	List all CH1 + CH2 apps for a BG/env
GET	<code>/api/applications/summary</code>	Aggregated count by status
GET	<code>/api/applications/:id</code>	Single application detail
POST	<code>/api/applications/:id/start</code>	Start a stopped application
POST	<code>/api/applications/:id/stop</code>	Stop a running application
POST	<code>/api/applications/:id/restart</code>	Restart an application

API Manager

Method	Path	Description
GET	<code>/api/apis</code>	List API Manager instances
GET	<code>/api/apis/:id/policies</code>	List applied policies (with human-readable names)
GET	<code>/api/apis/:id/contracts</code>	List consumer contracts with client IDs
GET	<code>/api/apis/:id/alerts</code>	List API-level alerts

Exchange

Method	Path	Description
GET	<code>/api/exchange/assets</code>	Search Exchange assets (query, type, orgId, limit)

Organizations & Environments

Method	Path	Description
GET	<code>/api/organizations</code>	Business group hierarchy
GET	<code>/api/environments</code>	Environment list for an org

Metrics

Method	Path	Description
GET	<code>/api/metrics/summary</code>	Aggregated throughput and error metrics

CPS

Method	Path	Description
GET	<code>/api/cps/:appName</code>	Config Property Server values for an application

Health / Ping

Method	Path	Description
POST	<code>/api/ping</code>	Run health check against one or more apps

Ping Test Logic

The `POST /api/ping` endpoint performs endpoint discovery and health verification for each submitted application.

Endpoint Resolution

CloudHub 1.0:

None

Base URL = `https://{appName}.api.sfdcibt.net`

CloudHub 2.0:

1. The frontend supplies the `ch2IngressUrl` from the deployment descriptor.
2. If the URL is comma-separated (multiple ingress endpoints), the list is split and the first URL that does **not** contain `internalapi` is preferred.
3. Falls back to the first URL if all contain `internalapi`.

Path Discovery Loop

Once the base URL is resolved, the backend tries each path in order until one returns a usable HTTP response:

```
None
1. /api/v1/ping
2. /api/v2/ping
3. /api/ping
4. /ping
```

Responses containing the text "`No listener for endpoint`" are treated as misses and the loop continues to the next path. This prevents false positives from Mule's default 404 handler.

Request Headers Sent

```
None
client_id:      <from request body, if provided>
client_secret:  <from request body, if provided>
x-transaction-id: <random UUID generated per ping run>
```

SSL

`rejectUnauthorized: false` is set on the HTTPS agent to handle internal CA-signed certificates common in enterprise CloudHub deployments (e.g., `*.sfdcbt.net`).

Response

```
JSON
{
  "success": true,
  "status": 200,
  "latency": 142,
```

```

"endpoint": "https://my-app.api.sfdcibt.net/api/v1/ping",
"path": "/api/v1/ping",
"payload": "{ \"status\": \"UP\" }",
"attempts": [
  { "path": "/api/v1/ping", "status": 200, "ok": true }
]
}

```

On failure:

```

JSON
{
  "success": false,
  "error": "ECONNREFUSED",
  "attempts": [
    { "path": "/api/v1/ping", "error": "ECONNREFUSED" },
    { "path": "/api/v2/ping", "error": "ECONNREFUSED" },
    { "path": "/api/ping", "error": "ECONNREFUSED" },
    { "path": "/ping", "error": "ECONNREFUSED" }
  ]
}

```

Known Limitations

Limitation	Detail
CloudHub 2.0 ingress URL	The CH2 ingress URL must be available in the deployment descriptor returned by the ARM API. If it is absent, the ping test cannot resolve the base URL.
CPS availability	CPS access depends on your Anypoint Platform entitlements. If CPS is not enabled for your organisation, the CPS Config tab will return an empty or error state.
Session expiry	The Anypoint bearer token has a default TTL of 60 minutes. If the token expires mid-session, you will be redirected to the login page.

Limitation	Detail
Rate limiting	Bulk operations (All Organizations aggregation, bulk ping) make many concurrent Anypoint API calls. Heavy use may trigger Anypoint Platform rate limits.
HTTPS certificates	<code>rejectUnauthorized: false</code> is used for ping health checks to support internal CA certs. Do not use this setting in environments where endpoint authenticity must be verified.
Read-only for most operations	Start/Stop/Restart are the only write operations. All other pages are read-only views of the Anypoint Platform state.
No real-time updates	The dashboard does not use WebSockets or polling. Refresh the page or navigate away and back to get updated data.

Contributing

1. Fork the repository.
 2. Create a feature branch: `git checkout -b feature/your-feature-name`
 3. Commit your changes following conventional commits: `feat:`, `fix:`, `docs:`, `refactor:`
 4. Open a Pull Request describing what was changed and why.
-

References

- [Anypoint Platform Documentation](#)
 - [Anypoint Runtime Manager API](#)
 - [Anypoint API Manager API](#)
 - [Anypoint Exchange API](#)
 - [DataWeave 2.0 Reference](#)
-

Built for MuleSoft platform operators and integration teams. For issues or feature requests, open a GitHub issue.